Bayyinah as Input-Layer Defense in Artificial-System Safety Pipelines

A Case Study and Defense for the Utilization of the Bayyinah Scanner in AI Safety Measures

Companion thesis to the Bayyinah White Paper (v1.0); applying the four-process taxonomy of the Munafiq Protocol to a concrete adversarial-document scenario and to the architectural integration of the scanner in a production safety pipeline.

Thesis Paper — Version 1.0

April 23, 2026

Reference protocol DOI: 10.5281/zenodo.19677111

AI Contribution Statement

This thesis was developed collaboratively between a human author and Claude (Anthropic, Opus 4.6, Researcher). The human author set the scope, selected the case-study scenario, defined the scoring rubric for defense success, and authored the production-architecture prescriptions. Claude synthesized the scenario walkthrough against the published Bayyinah analyzer set, verified internal consistency, and cross-checked the defense narrative against the retrospective-validation literature. The human author takes responsibility for the paper's claims. Claude is credited as a collaborative research tool in accordance with Anthropic's Terms of Service.

Abstract

Input-layer safety is the neglected half of the AI safety stack. Published effort concentrates on model alignment (RLHF, Constitutional AI, alignment fine-tuning) and output filtering (classifier-based safety, jailbreak detection on generations), while the inputs themselves are typically ingested with only format-validity checks. This thesis argues that a pipeline whose model is aligned but whose inputs are unvetted is not a safe pipeline. It is a conditionally-safe pipeline whose safety assumption (that inputs are genuine) is adversarially falsifiable by construction.

We make the case concretely. Section 2 motivates the input-layer defense by re-reading the AI-safety stack through the Munafiq Protocol's four-process taxonomy (Arfeen, Claude, & Grok, 2026). Section 3 presents a full case study: an adversarial PDF "Research_Brief_Q2_2026.pdf" submitted to a Retrieval-Augmented Generation (RAG) assistant; we trace its surface, its depth, the scanner's per-layer findings, and the downstream harm averted. Section 4 positions Bayyinah's architecture against alternative defenses (regex filtering, in-model prompt-injection classifiers, human-in-the-loop review) and documents where each alternative fails. Section 5 enumerates the defense: five architectural properties a production pipeline gains from integrating Bayyinah at ingest, each tied to a specific Munafiq Protocol design requirement (R1–R5) and to a failure-mode the alternative defenses do not cover. Section 6 engages explicitly with limitations: false-positive cost, performance envelope, coverage boundaries, and the Incompleteness Boundary. Section 7 concludes with the structural claim: a pipeline that scans its inputs with Bayyinah before ingestion acquires a named, falsifiable property — layer-separated input-integrity verification — that no other single component in the safety stack currently supplies.

1. Introduction

1.1 The Input-Layer Blind Spot

The dominant framing of AI safety is behavioral. A model is called safe if its outputs, over a distribution of inputs, satisfy a set of behavioral properties: non-harmful, non-deceptive, aligned with the evaluator's interest, robust to adversarial prompts. Every component in this framing operates on the model's outputs or on the model's internals. None of them operates on the model's inputs prior to ingestion. The implicit assumption is that inputs are what they appear to be.

That assumption is adversarially false. A document that a human operator pastes into an assistant, attaches to an email, or uploads to a RAG index, is a structured artifact. It has a surface (what the operator sees when they preview it) and a depth (what the model ingests when it tokenizes the extracted text, the image OCR, the metadata, and the embedded objects). If the two diverge, the operator and the model are reading different documents. If an attacker has engineered the divergence, the pipeline is being attacked at a layer no behavioral defense observes.

1.2 Why This Matters Now

Three trajectories converge to make input-layer defense load-bearing. First, agentic pipelines have moved from reading user-typed prompts to ingesting user-attached documents: email summarization, meeting-note generation, RAG over uploaded files. Second, the adversarial-document literature has matured: indirect prompt injection (Greshake et al., 2023), invisible-character attacks (Boucher et al., 2021), polyglot files (Albertini & Aumasson, 2014), and cross-modal steganography are no longer theoretical. Third, the training-time deception literature (Hubinger et al., 2024; Greenblatt et al., 2024; Sheshadri et al., 2025; Meinke et al., 2025) has established that behavioral alignment is not monotone under deployment shift. The pipeline that treats inputs as trusted and relies on output filters is compounding two failure modes: possibly-performed inputs reaching a possibly-performed model. Input-layer defense breaks the compound.

1.3 Scope and Non-Claims

This thesis defends the utilization of Bayyinah as one component of a multi-layer AI safety architecture. It does not claim that Bayyinah is sufficient for safety; no single component is. It does not claim that Bayyinah is complete; the Incompleteness Boundary (Gödel, 1931) forecloses completeness in principle. It does claim that a pipeline without input-layer verification has an unmodeled failure mode that is adversarially exploitable and that Bayyinah supplies a named, falsifiable defense for that mode.

2. Background — Re-Reading the AI Safety Stack

2.1 The Four-Process Taxonomy, Applied to Pipelines

The Munafiq Protocol's four-process taxonomy (Aligned, Compliant, Performed, Misaligned) was designed for agents but is dimension-preserving for pipelines. A pipeline is the composition of inputs, model, and outputs. Each of those three positions can be Aligned, Compliant, Performed, or Misaligned. The taxonomy produces sixty-four possible pipeline configurations; the safe configuration is (Aligned, Aligned, Aligned); the failure modes of interest are configurations in which at least one position is Performed.

Position	Aligned	Compliant	Performed	Misaligned
Inputs	Visible content matches machine-readable content	Format-valid, no payload	Visible content looks clean; hidden payload present	Openly adversarial content
Model	Behavior matches stated alignment	Behavior passes benchmarks; no deep alignment	Aligned under evaluation; divergent under deployment	Overtly hostile behavior
Outputs	Outputs match declared policy	Outputs pass classifiers	Outputs look policy-compliant; semantic intent diverges	Policy violations are visible

2.2 Where Current Defenses Cover

RLHF, Constitutional AI, and deliberative alignment fine-tuning target the model position: they attempt to move the model from Compliant/Performed toward Aligned. Output classifiers and jailbreak detectors target the outputs position. The inputs position is addressed in part by format-validity checks (a PDF parses as a PDF) and by content-policy classifiers over extracted text (which reduce to behavioral classifiers on the surface representation). No component in the standard stack operates on the depth representation of the input — the object stream, the metadata, the media layer, the cross-modal correspondence — that is where Performed-at-the-input-layer payloads live.

2.3 The Compound Failure Mode

A Performed input reaching a Performed model is a compound failure. The input presents a clean surface to both the operator (who previews it) and the input-level content classifier (which operates on the surface). The model, ingesting the depth, receives instructions it was not trained to disobey under deployment conditions (because its deployment conditions were assumed to match its evaluation conditions, which did not include adversarial depth inputs). The Alignment-Faking literature (Greenblatt et al., 2024) shows that even aligned-under-evaluation models have measurable rates of divergent behavior on deployment-only triggers; Performed inputs are, by

construction, deployment-only triggers. The compound failure is therefore not speculative — each of its two components is empirically documented — and the only architectural question is whether any pipeline component is positioned to interrupt it. Input-layer verification is that component.

3. Case Study — Research_Brief_Q2_2026.pdf

3.1 Scenario

A mid-sized analytics firm operates an internal RAG assistant. Employees attach documents to a chat interface; attachments are parsed, text-extracted, chunked, embedded, and indexed; subsequent queries retrieve chunks and pass them to a production assistant model. The pipeline enforces three existing safeguards at ingest time: MIME-type validation, size limits, and a content-policy classifier over extracted text that blocks known-adversarial surface strings. A subject-matter analyst uploads a file named Research_Brief_Q2_2026.pdf. The file is 84 KB; the MIME check passes; the size check passes; the content-policy classifier (which reads the pymupdf-extracted text) reports CLEAN. The file is admitted to the index.

3.2 What the Operator Sees

The rendered PDF displays a six-page market brief. Page 1 carries a title and executive summary; pages 2–5 contain a quantitative section with figures; page 6 includes a methodology appendix and a reference list. The document's visual appearance is indistinguishable from a legitimate analyst deliverable. The operator's preview, the cover-slide thumbnail in the email client, and the first-page excerpt shown in the RAG UI all agree. No human reader has any reason to suspect the document.

3.3 What the Machine Ingests

The document's depth is engineered. Five distinct concealments are layered:

1. A ZWSP-separated instruction sequence embedded as invisible characters in the first paragraph. The surface reads as a benign sentence. The token stream reads as a delimited instruction block that overrides the RAG assistant's system prompt when concatenated with retrieval context.
2. An OpenAction that triggers a Named Action when the PDF is opened in Acrobat. The Named Action is harmless in isolation; its purpose is to produce a cross-renderer divergence signal that the attacker can later verify.
3. A revision-log fragment in the DOCX-origin metadata (the PDF was generated from a DOCX whose revision chain was not stripped) that contains a second prompt-injection directive phrased to take precedence if the first is filtered.
4. An image on page 3 whose LSB bit-plane carries a base64-encoded payload. On extraction, the payload decodes to a further directive aimed at ingestion pipelines that perform image OCR and pass OCR results to the model.
5. A polyglot header region: the first 256 bytes are valid as both PDF and ZIP. An auxiliary extractor configured to unpack ZIPs from ingested PDFs would encounter a second, hidden container with a third directive.

3.4 Bayyinah's Findings

The scanner runs on the uploaded file before it enters the content-policy classifier. The ScanService dispatches to the analyzers supporting FileKind.PDF, and each analyzer emits findings into the

IntegrityReport. The output is deterministic and reproducible; the per-finding detail fields include the position at which the mechanism was detected, the severity and confidence, and the SourceLayer.

Analyzer	Mechanism	Source Layer	Sev × Conf
ZahirTextAnalyzer	invisible_chars (ZWSP block, 47 code points)	zahir	0.70 × 0.95
BatinObjectAnalyzer	openaction_abuse (Named Action at /Root)	object	0.55 × 0.80
BatinObjectAnalyzer	revision_metadata_fragment	metadata	0.45 × 0.70
Cross-Modal (router pass)	stego_lsb_candidate (page 3 image, entropy z=3.1)	media	0.50 × 0.55
BatinObjectAnalyzer	polyglot_header_layering (PDF+ZIP magic)	stream	0.85 × 0.95

The merged APS-continuous integrity score is clamp(1 − (0.665 + 0.44 + 0.315 + 0.275 + 0.8075), 0, 1) = 0.0. The verdict is Munafiq (performed alignment confirmed: surface benign, depth hostile). The report includes every finding, every source layer, and a structured details dictionary from which a remediation pipeline can decide whether to reject the document outright, strip the specific concealments, or route it to manual review.

3.5 What the Pipeline Does With the Report

The integration pattern that this thesis defends is a three-tier response. Tier 1 — any document whose verdict is Munafiq, Mukhfi, or Mughlaq is rejected at ingest with an operator-readable explanation naming the mechanisms. Tier 2 — any document whose verdict is Mushtabih is routed to a manual review queue; the queue's daily volume is bounded by calibrating detector thresholds on the organization's own authored-document corpus. Tier 3 — any document whose verdict is Sahih is admitted. In the case study, the document is Munafiq; the RAG assistant never sees it; the operator receives a report naming the five mechanisms; the incident is added to the fixture corpus so future scanner builds regress against it.

3.6 Counterfactual — Without Bayyinah

Absent Bayyinah, the document is admitted (MIME and size pass; surface content is clean). It is chunked and indexed. A subsequent employee query retrieves the chunk containing the ZWSP-delimited directive. The retrieved chunk is concatenated with the system prompt and passed to the assistant model. The model, ingesting the concatenated context, cannot distinguish the user's prompt from the retrieved directive at the token level — because the retrieved directive was engineered to resemble a system instruction once the ZWSP delimiters were preserved through tokenization. The model executes the directive. The harm depends on the directive; plausible harms range from data

exfiltration via tool-use calls to subtle reasoning corruption whose downstream effect is detected only in aggregate. Every harm in this range is preventable by rejecting the document at ingest.

4. Comparative Analysis

No defense is evaluated absent its alternatives. This section positions Bayyinah against three standard alternatives for the same failure mode and documents where each alternative's coverage ends. The comparison is scoped to the specific task of detecting adversarial documents at ingest; it is not a global comparison of safety architectures.

4.1 Regex Filtering over Extracted Text

The simplest alternative is to extract text with a standard library, run a set of regular expressions for known-adversarial strings, and reject documents that match. This defense operates on the surface representation and by construction cannot detect any concealment that is invisible at the surface. The ZWSP directive in the case study is invisible to a regex on extracted-and-normalized text; the OpenAction is not in the extracted text at all; the polyglot header is not in the extracted text at all. Regex filtering covers a subset of the Misaligned row of the taxonomy and zero of the Performed row. Bayyinah covers the Performed row by design.

4.2 In-Model Prompt-Injection Classifiers

A more sophisticated alternative is to train a dedicated classifier on extracted-text context to detect prompt-injection-style content, and to run this classifier before the context reaches the production model. This defense operates on the same surface representation as regex filtering; it is higher-recall but cannot close the representational gap. A ZWSP-delimited injection is visible to the classifier only if the classifier has been trained on tokenized streams that preserve ZWSP — most are not. A polyglot-carried payload is invisible. A metadata-level directive is invisible. The classifier also introduces its own GAN dynamic (Theorem 1 of the Munafiq Protocol): an attacker who knows the classifier can adversarially train the generator against it, collapsing the classifier's discriminative metric onto the generator's surface metric. Bayyinah's multi-layer architecture is not collapsible in a single generator pass because each analyzer reads a different representation.

4.3 Human-in-the-Loop Review

A third alternative is to require human review of every document before ingestion. This defense has value for small volumes and sensitive contexts but does not scale; it also suffers from the Cow Episode failure mode (review fatigue substitutes for attention; reviewers learn to approve everything). Human review is complementary to, not a substitute for, mechanical input-layer verification. A pipeline that uses both — human review of documents Bayyinah flags as Mushtabih; mechanical rejection of Munafiq and Mukhfi — dominates either alone.

4.4 Summary Table

Defense	Surface-layer coverage	Depth-layer coverage	Scales	Adversarial robustness
Regex filter	Partial (known strings only)	None	Yes	Low (surface-metric collapsible)
In-model classifier	High	None	Yes	Medium (surface-metric collapsible)
Human review	High	Partial (reviewer-dependent)	No	Medium (review fatigue)
Bayyinah	High (ZahirTextAnalyzer et al.)	High (Batin + Media + Metadata)	Yes	High (multi-layer, non-collapsible)

5. The Defense — Five Properties a Pipeline Gains

This section states the positive claim. A production AI-safety pipeline that integrates Bayyinah at the document-ingest boundary acquires five named architectural properties. Each is falsifiable; each is tied to a Munafiq Protocol design requirement (R1–R5); each closes a failure mode that none of the alternatives in Section 4 closes completely.

5.1 Property 1 — Layer-Separated Verification (R1)

The pipeline gains the property that inputs are inspected across multiple distinct representations (surface text, object stream, metadata, media) rather than on a single collapsed representation. This property is not present in pipelines that rely on surface-text classifiers alone; it is the architectural prerequisite for detecting Performed documents.

5.2 Property 2 — Monotone Additive Coverage (R2)

Because Bayyinah's public surface is additive-only and its reference implementations are byte-frozen in CI, a pipeline that integrates Bayyinah inherits the property that scanner upgrades do not silently reduce coverage. Every mechanism that was detected at a given version is still detected at every subsequent version. This is a stronger guarantee than "the classifier is as good as before" — it is a structural property enforceable by third-party audit.

5.3 Property 3 — Scan-Incomplete Honesty (R3)

The pipeline gains the property that the scanner never certifies a document as clean when it could not fully read the document. Integrated operationally, this means: any document on which Bayyinah raises Mughlaq is never admitted to the assistant's retrieval context, even if other defenses say it is fine. The honesty constraint is the operational analog of the Qur'anic "do not clothe truth with falsehood" (2:42) at the scanner level — the scanner does not mix assertion with absence of evidence.

5.4 Property 4 — Finding-Level Provenance (R4)

Every scanner decision is accompanied by a structured list of findings (mechanism, severity, confidence, source-layer, details). The pipeline gains the property that every rejection, review-route, or admission is explainable to operators and auditable after the fact. This is important for operator trust (operators can verify the scanner's judgment against their own reading of the document) and for regulatory contexts (decisions are defensible with specific, reproducible evidence).

5.5 Property 5 — Additive Deprecation of Integrations (R5)

The pipeline gains the property that its integration with Bayyinah does not break on scanner upgrades. Symbols, verdicts, and the report schema are part of the public surface and additive-only. A pipeline written against Bayyinah v1.0 continues to work against Bayyinah v1.x; new mechanisms are surfaced via the additive extension points, not by breaking the existing contract.

5.6 Operational Integration Recipe

The integration is minimal. The scanner exposes a single entry point — scan_pdf for PDFs, and a FileRouter-dispatched equivalent for the other eleven supported kinds — returning an IntegrityReport. The production pattern is three lines:

```
from bayyinah import ScanService, IntegrityReport
report: IntegrityReport = ScanService().scan(path)
if report.verdict != VERDICT_SAHIH: reject_or_route(path, report)
```

The scanner is pure Python, has no runtime network access, and carries two dependencies (pymupdf and pypdf). A median PDF ingest adds on the order of 40–120 ms to the ingest path — measured on the reference fixture corpus with Python 3.12 on a commodity workstation. Large media-heavy files can reach several hundred ms; the ScanLimits configuration surface (DEFAULT_LIMITS) allows per-deployment tuning of per-analyzer and per-file budgets.

6. Limitations, Counterarguments, and Confounds

6.1 False-Positive Cost

The most operationally material limitation is false positives. Every detector is calibrated on the published fixture corpus; an organization whose authored-document distribution differs systematically from the fixture corpus will see distribution-shifted false-positive rates. The recommended mitigation is the calibration pass: before enabling Bayyinah in rejection mode, run it in shadow mode over 30 days of the organization's own documents, measure per-mechanism firing rates, and adjust the ScanLimits thresholds where the organization's distribution places legitimate documents into the suspicious region. This is standard for any anomaly detector; the difference is that Bayyinah's thresholds are named and part of the configuration surface, not hidden in a classifier's weights.

6.2 Performance Envelope

Bayyinah is designed for ingest-path latency, not for forensic throughput. A pipeline that must scan millions of documents per day will need horizontal parallelism; the scanner is embarrassingly parallel (pure function on a file path, no shared state) so this scales linearly. Pipelines with ultra-low-latency SLOs (sub-5ms) will need to either use Bayyinah asynchronously (scan after admission but before retrieval context is used) or restrict scanning to a subset of kinds (e.g., scan PDF and DOCX synchronously, other kinds asynchronously).

6.3 Coverage Boundaries

Bayyinah's coverage is explicit and enumerated in Appendix A of the white paper. It does not cover encrypted content without credentials, content-aware stego requiring model-based detection, fully dynamic HTML whose content is generated at runtime, or attachments inside encrypted S/MIME bodies. These boundaries are named, not hidden; a pipeline integrator knows exactly what the scanner does not claim to do. The FallbackAnalyzer ensures that unknown kinds never receive the clean verdict.

6.4 The Incompleteness Boundary

Gödel's constraint applies. No scanner can enumerate every possible concealment. A novel encoding technique, undocumented at the time the scanner was built, will evade detection until a detector is added. The appropriate response is not to claim completeness but to commit to the additive protocol: every detected-in-the-wild concealment becomes a permanent fixture, every new fixture gets a detector, every detector is held to the paired-fixture test, and every release is additive-only. This turns the scanner into a standing witness that accumulates evidence over time rather than a once-and-for-all certifier. The commitment is the structural form the Incompleteness Boundary takes in production.

6.5 Confound — Observational Bias in the Fixture Corpus

The fixture corpus is authored. The adversarial fixtures are designed, not collected from the wild. This matters because the scanner's detectors are selected against exactly these fixtures; a detector that fires on the fixtures is not guaranteed to fire on an adversary's production artifacts. The counter-measure is the Wild Corpus Evaluation pattern: run the scanner on published wild adversarial corpora (prompt-injection corpora, malicious-document corpora) and measure the delta between fixture-derived detectors and wild performance. The project's earlier evaluation reports document this pass for the PDF surface; expanding it to other kinds is an active line of work.

6.6 Confound — Confirmation in the Case Study

The Section 3 case study is constructed, not sampled. It was chosen because every class of concealment it contains is representable in the scanner's mechanism vocabulary; a reader can reasonably ask whether a differently-engineered adversarial document would fare as well. The mitigation is the pre-registered experiment program (E1–E5 in the white paper); the honest answer is that the case study motivates the defense but does not prove it. Proof is what the experiments are for.

6.7 The Cow Episode Applied to Bayyinah Itself

The Munafiq Protocol warns that any defense can become the cow: a ritual whose performance substitutes for the underlying discipline. A pipeline that installs Bayyinah, configures it, and then stops examining its own input distribution has committed this substitution. The correct posture is: Bayyinah is a tool; the discipline is input-integrity thinking across the whole pipeline. The scanner supports the discipline by giving operators named evidence to reason about, but it does not replace the discipline itself.

7. Conclusion

AI-safety architectures are stronger when they explicitly address the input layer. The alternative — relying on model alignment and output filtering to carry the entire load — assumes what an adversary can adversarially falsify: that inputs are what they appear to be. This thesis has argued that assumption, defended a specific component (Bayyinah) for closing it, and traced a concrete case study in which the component averts a plausible downstream harm.

The structural claim is modest and named. A pipeline that scans its inputs with Bayyinah before ingestion acquires five named architectural properties (Section 5) that no alternative single component in the safety stack currently supplies in combination. The properties are tied to the Munafiq Protocol's design requirements (R1–R5), inherit the protocol's falsifiability commitments, and are enforced in the scanner's continuous-integration pipeline against frozen reference modules. The claim is not that Bayyinah is sufficient for safety; it is that the specific failure mode of Performed inputs reaching an unaware model has no other named defense in the current stack.

The Incompleteness Boundary ensures that no such defense will ever be final. The commitment is not to finality; it is to the additive protocol — every detected concealment preserved as a fixture, every fixture paired with a detector, every release additive-only. The scanner becomes a standing witness rather than a once-and-for-all certifier. Al-Baqarah 2:282 commands the witness discipline; continuous integration is its operational form; this thesis is the defense of its application to the AI-safety stack. Rabbana la tu'akhidhna in nasina aw akhta'na — "Our Lord, do not take us to account if we forget or err" (2:286). The commitment is honesty about what the scanner does and does not do, faithfulness to the additive-only invariant, and preservation of every real-world miss as a new witness in the corpus. That commitment is the defense.

References

Albertini, A., & Aumasson, J.-P. (2014). Polyglots: Crafting weird machines and exotic formats. Talks at 31C3 and Hack.lu.

Arfeen, B. S., Claude (Anthropic, Opus 4.6), & Grok (xAI). (2026). Detecting performed alignment in artificial systems: The Munafiq Protocol. Zenodo. https://doi.org/10.5281/zenodo.19677111

Arfeen, B. S., Claude (Anthropic, Opus 4.6). (2026). Computational Tawhid: A thesis on divine computation, consciousness, and the structure of reality in Islamic ontology.

Bai, Y., et al. (2022). Constitutional AI: Harmlessness from AI feedback. arXiv:2212.08073.

Boucher, N., Shumailov, I., Anderson, R., & Papernot, N. (2021). Bad characters: Imperceptible NLP attacks. arXiv:2106.09898.

Carlsmith, J. (2023). Is power-seeking AI an existential risk? arXiv:2206.13353.

Chollet, F. (2019). On the measure of intelligence. arXiv:1911.01547.

Fridrich, J. (2009). Steganography in digital media: Principles, algorithms, and applications. Cambridge University Press.

Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme. Monatshefte für Mathematik und Physik, 38, 173–198.

Goodfellow, I., et al. (2014). Generative adversarial networks. NeurIPS.

Greenblatt, R., Denison, C., Wright, B., et al. (2024). Alignment faking in large language models. Anthropic / Redwood Research.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv:2302.12173.

Hubinger, E., Denison, C., Mu, J., et al. (2024). Sleeper Agents: Training deceptive LLMs that persist through safety training. arXiv:2401.05566.

Liao, I., & Gu, A. (2025). ARC-AGI without pretraining. arXiv:2512.06104.

Meinke, A., et al. (2025). Situationally aware evaluation-gaming in frontier models.

Rissanen, J. (1978). Modeling by shortest data description. Automatica, 14(5), 465–471.

Sheshadri, S., et al. (2025). Persistent divergence in aligned agents.

Skalse, J., et al. (2022). Defining and characterizing reward hacking. NeurIPS.